

Assignment 4: Matrix Multiplication Optimization

Group Members

Behroz Karim – 2502071
Hasnain Ajmal – 2502065
Hugo Pérez Muñoz – 2501103
Talha Rizwan – 2503934
Max Nummila – 2202236

May 2026

1 System Information:

Property	Value
Architecture	x86_64
CPU	13th Gen Intel Core i5-13420H (Raptor Lake-H)
Cores	6 (12 hardware threads, 2 threads per core)
Max CPU clock	4.6 GHz turbo
SIMD Support	AVX2 (8-wide vector instructions for single-precision float)
Total Hardware Threads	12
GPU	NVIDIA GeForce RTX 3050 6 GB Laptop GPU
GPU architecture	Ampere, SM 8.6, 20 SMs
GPU memory	6 GB GDDR6, 168 GB/s peak bandwidth
GPU host link	PCIe Gen4 ×4
Host RAM	24 GB DDR5-5200 SODIMM (WSL2 sees 11 GB by default)
OS	Ubuntu 22.04 in WSL2 on Windows 11
NVIDIA driver	596.36
CUDA toolkit	12.4
Host compiler	GCC 11

2 Introduction

The assignment was about implementation and optimization of single-precision dense matrix multiplication in C++. This report documents this and compares the three variants:

1. A **naive** textbook implementation as the baseline.
2. A **cache-tiled** (blocked) variant intended to improve data locality.
3. A **parallel** variant using **OpenMP** to distribute work across CPU threads.

In this report we explain the implementation and report wall-clock timings and speedups over the naive baseline. The report describes not only the final solution but also the iterative process that led to it.

3 Implementation Overview

Matrix convention throughout the implementation: A is $m \times n$, B is $n \times p$, and C is $m \times p$.

3.1 naive_matmul

The naive implementation is the textbook triple-nested loop. A local `sum` accumulator is used so the innermost loop avoids repeatedly reading and writing `C[i*p+j]` from memory, the running total is held in a register and written out only once per output element.

```
void naive_matmul(float *C, float *A, float *B, uint32_t m, uint32_t n,
uint32_t p) {
    for (uint32_t i = 0; i < m; ++i) {
        for (uint32_t j = 0; j < p; ++j) {
            float sum = 0.0f;
            for (uint32_t k = 0; k < n; ++k) {
                sum += A[i * n + k] * B[k * p + j];
            }
            C[i * p + j] = sum;
        }
    }
}
```

3.2 blocked_matmul

The blocked implementation uses the six-loop tile structure from the assignment pseudocode. The outer loops (`ii`, `jj`, `kk`) iterate over `block_size` $\times$ `block_size` tiles so that each tile's working set is intended to fit in cache. The inner loops, however, use the $i \rightarrow k \rightarrow j$ order rather than the pseudocode's $i \rightarrow j \rightarrow k$. We will discuss the reasoning for this below. `std::min(...)` handles the tile boundaries for matrices whose dimensions are not a multiple of `block_size` (cases 2, 5, 7, and 9). Because each tile accumulates into `C` across the `kk` loop, `C` is zero-initialized at the start of the call.

```
void blocked_matmul(float *C, float *A, float *B, uint32_t m, uint32_t
n, uint32_t p, uint32_t block_size) {
    for (uint32_t i = 0; i < m * p; ++i) {
        C[i] = 0.0f;
    }
    for (uint32_t ii = 0; ii < m; ii += block_size) {
        for (uint32_t jj = 0; jj < p; jj += block_size) {
            for (uint32_t kk = 0; kk < n; kk += block_size) {
                uint32_t i_max = std::min(ii + block_size, m);
                uint32_t j_max = std::min(jj + block_size, p);
                uint32_t k_max = std::min(kk + block_size, n);
                for (uint32_t i = ii; i < i_max; ++i) {
                    for (uint32_t k = kk; k < k_max; ++k) {
                        float a_ik = A[i * n + k];
                        for (uint32_t j = jj; j < j_max; ++j) {
                            C[i * p + j] += a_ik * B[k * p + j];
                        }
                    }
                }
            }
        }
    }
}
```

3.3 parallel_matmul

The parallel implementation parallelizes the outer i loop using `#pragma omp parallel for`. Each thread is assigned a disjoint set of rows of C , it first zeros those rows, then performs the $i \rightarrow k \rightarrow j$ accumulation. Because the rows are disjoint across threads, the kernel is data-race free without any atomics, locks, or `reduction` clauses. Having each thread also do the zero-initialization of its own rows provides a small additional benefit by establishing NUMA “first-touch” memory affinity.

```
void parallel_matmul(float *C, float *A, float *B, uint32_t m, uint32_t
n, uint32_t p) {
    #pragma omp parallel for
    for (uint32_t i = 0; i < m; ++i) {
        for (uint32_t j = 0; j < p; ++j) {
            C[i * p + j] = 0.0f;
        }
        for (uint32_t k = 0; k < n; ++k) {
            float a_ik = A[i * n + k];
            for (uint32_t j = 0; j < p; ++j) {
                C[i * p + j] += a_ik * B[k * p + j];
            }
        }
    }
}
```

3.4 Supporting code

Two small helpers, `read_matrix` and `write_matrix`, handle the assignment’s ASCII format (a “<rows> <cols>” header line followed by whitespace-separated floats). The `validate_result` function compares the written `result.raw` against the supplied `output.raw` element-by-element with an absolute tolerance of 1×10^{-2} (to accommodate the two-decimal-place reference printing) and a relative tolerance of 1×10^{-3} (to absorb accumulated floating-point error in larger dot products).

4 Performance Measurement

4.1 Timing Methodology

Wall-clock time is measured with `omp_get_wtime()`, which provides high-resolution timing suitable for sub-millisecond intervals. To reduce run-to-run variability (the README’s “Tips for Success” section explicitly recommends this), each implementation is invoked five times in a tight loop and the average is reported. The number of OpenMP threads is set externally via `OMP_NUM_THREADS`; an empirical sweep on case 6 found `OMP_NUM_THREADS=8` to be the best fit for this CPU’s six physical cores, so this value was used for the headline measurements.

Validation is performed independently for each of the three implementations after every test case, ensuring that any timing reported was for a numerically correct result. The first run is slower than the others probably because CUDA does some one-time setup work the very first time a kernel is launched (context creation etc). That’s why in our discussion below we ignore the 1st run and discuss based on results from runs 2 and 3.

5 The Optimization Journey

5.1 The first attempt and what it revealed

The initial implementation was literally based on the pseudocode provided in the assignment’s readme. `blocked_matmul` used the inner loop order $i \rightarrow j \rightarrow k$, and `parallel_matmul` used the same body wrapped in `#pragma omp parallel for`. Building with the supplied `CMakeLists.txt` and running all ten test cases produced numerically correct outputs for every case, but the timing picture was disappointing on two fronts: the blocked variant was *slower* than the naive baseline across essentially every case, and the parallel variant, despite running on eight threads, was also returning no significant speedups. This forced an investigation into *why* the optimizations were producing so little benefit.

5.2 Diagnosis

There were two major problems:

5.2.1 The matrices already fit in cache

Cache blocking reduces capacity misses by ensuring that the working set fits in a chosen cache level. The technique therefore yields its benefits when the matrices being multiplied are **larger** than the cache. In our case, each test matrix occupies at most 300 KB, and even three such matrices (A, B, C) fit comfortably inside the i5-13420H’s 1.25 MB-per-core L2 cache. With virtually no capacity misses to eliminate, blocking introduces overhead, six nested loops, boundary-clipping arithmetic via `std::min`, and zero-initialization of C for the accumulating kernel, without producing the cache-miss reduction it normally provides.

This observation alone explained why our blocked variant could never substantially outperform naive on this benchmark suite: the optimization was operating on a workload outside its intended regime.

5.2.2 The inner loop ordering was suboptimal

While the cache-fit problem could not be addressed without changing the workload, the inner loop ordering was found to be a second, *fixable* source of inefficiency.

In $i \rightarrow j \rightarrow k$ order, the innermost statement is:

```
C[i*p+j] += A[i*n+k] * B[k*p+j];
```

with k varying. Each iteration advances `B[k*p+j]` by p floats, a column-stride access on a row-major matrix. This stride defeats the hardware prefetcher (which is tuned for sequential access) and prevents the compiler’s auto-vectorizer from emitting SIMD instructions, since vectorization requires unit-stride memory accesses. The same flaw appears in the naive implementation, but the blocked variant compounds it with additional per-iteration memory traffic to C (read-modify-write via `+=`).

5.3 The $i \rightarrow k \rightarrow j$ loop reordering

After reviewing standard cache-optimization techniques for matrix multiplication, we adopted the well-known transformation of swapping the innermost two loops to produce the $i \rightarrow k \rightarrow j$ ordering. The reorganized inner triple hoists `A[i*n+k]` out of the innermost loop (it is now invariant in j) and turns the innermost statement into a classic SAXPY-style operation:

```

for (i = ii; i < i_max; ++i)
  for (k = kk; k < k_max; ++k) {
    float a_ik = A[i*n+k];
    for (j = jj; j < j_max; ++j)
      C[i*p+j] += a_ik * B[k*p+j];
  }

```

The access pattern in the innermost loop changes substantially:

Reference	i-j-k order (stride)	i-k-j order (stride)
A[i*n+k]	1 (varies with k)	invariant, hoisted to a register
B[k*p+j]	p (column-stride)	1 (sequential)
C[i*p+j]	invariant in k	1 (sequential)

Two consequences follow. First, the hardware prefetcher operates correctly, because both **B** and **C** are now walked one full row at a time. Second, the compiler can auto-vectorize the inner loop into SIMD instructions, `c[j] += a * b[j]` is a textbook vectorizable pattern that modern compiler recognize. We will observe this in the results section.

The structural trade-off is that, with **j** as the innermost loop, the per-element local **sum** accumulator used in the naive implementation is no longer convenient, accumulation now occurs in **C** itself. This requires zero-initializing **C** at the start of each call and accepting one additional $m \times p$ memory-write pass. At the matrix sizes considered here, this cost is negligible compared to the throughput improvements unlocked by vectorization and prefetcher-friendly access.

The same transformation was applied to `parallel_matmul`, with each OpenMP thread first zeroing its assigned rows of **C** and then performing the **i-k-j** accumulation.

5.4 Block-size selection

We used the block size of 32. To confirm that the chosen `block_size = 32` was reasonable, the blocked variant was rerun with `block_size = 16` and `block_size = 64` on the larger test cases. There was no significant difference in the results across these three values, no value consistently outperformed the others. Since there was no significant difference, `block_size = 32` was retained. This null result is itself informative as it reinforces the conclusion of § 5.2, because when the working set fits entirely in cache, the choice of block size has little effect.

6 Results

All implementations produce results that pass validation against `data/<case>/output.raw` within the tolerances described in § 3.4. The results are presented in two stages: first under the build path the assignment specifies, then after an additional round of compiler-level tuning that further improved the speedups already obtained from the algorithmic changes.

6.1 Results from the supplied CMake build

Building with the supplied `CMakeLists.txt` (which enables OpenMP but sets no optimization flag) and running each test case with `OMP_NUM_THREADS=8` produced the results in Table 1.

Two observations stand out. First, the parallel implementation reaches up to $3.75\times$ speedup on case 1 and consistently exceeds $3\times$ on the medium and large cases, confirming that the $i \rightarrow k \rightarrow j$ reordering and the row-wise OpenMP parallelism work as intended. Second, the

Table 1: Results from the supplied CMake build with OMP_NUM_THREADS=8.

Case	Dimensions ($m \times n \times p$)	Time (s)			Speedup	
		Naive	Blocked	Parallel	Blocked	Parallel
0	$64 \times 64 \times 64$	0.000915	0.000913	0.000630	1.00×	1.45×
1	$128 \times 64 \times 128$	0.001891	0.001884	0.000504	1.00×	3.75×
2	$100 \times 128 \times 56$	0.001282	0.001333	0.000380	0.96×	3.38×
3	$128 \times 64 \times 128$	0.001970	0.001971	0.000889	1.00×	2.22×
4	$32 \times 128 \times 32$	0.000292	0.000249	0.000159	1.17×	1.84×
5	$200 \times 100 \times 256$	0.009277	0.009140	0.003738	1.01×	2.48×
6	$256 \times 256 \times 256$	0.032117	0.030844	0.011711	1.04×	2.74×
7	$256 \times 300 \times 256$	0.037422	0.035864	0.009984	1.04×	3.75×
8	$64 \times 128 \times 64$	0.001006	0.000917	0.000295	1.10×	3.41×
9	$256 \times 256 \times 257$	0.031400	0.031574	0.008730	0.99×	3.60×

blocked implementation hovers around $1\times$ across the board, exactly as predicted in §5.2: the test matrices fit entirely in L2 cache, so blocking has no capacity misses to eliminate and can do little more than match the naive baseline.

6.2 Looking beyond the algorithm

Two facts are clear from Table 1. Parallelization has delivered real gains, but blocking will not deliver further gains on this workload because the workload itself lies entirely within the cache hierarchy that blocking is designed to exploit. We researched a little about the compile time optimizations. The supplied `CMakeLists.txt` sets no optimization flag, leaving the build at GCC’s default behavior which engages neither the auto-vectorizer nor any of the loop transformations that would actually exploit the $i \rightarrow k \rightarrow j$ inner loop established in §5.3.

We tried different flag combinations. The flags we used were: `-O3`, `-march=native`, `-ffast-math`. All these flags have their own benefits for example `-O3` enables aggressive auto-vectorization.

The reason this matters specifically for the blocked variant is that the $i \rightarrow k \rightarrow j$ reordering of §5.3 had prepared an inner loop of the form `c[j] += a * b[j]` that is *ideal* for the auto-vectorizer, but only an aggressive compiler will actually emit the SIMD instructions to exploit it.

Running all ten cases under `-O3 -march=native -ffast-math` with `OMP_NUM_THREADS=8` produced the headline results in Table 2.

Table 2: Final results with `-O3 -march=native -ffast-math` and `OMP_NUM_THREADS=8`.

Case	Dimensions ($m \times n \times p$)	Time (s)			Speedup	
		Naive	Blocked	Parallel	Blocked	Parallel
0	$64 \times 64 \times 64$	0.000237	0.000152	0.000169	$1.56\times$	$1.41\times$
1	$128 \times 64 \times 128$	0.000825	0.000370	0.000163	$2.23\times$	$5.08\times$
2	$100 \times 128 \times 56$	0.000462	0.000248	0.000136	$1.86\times$	$3.40\times$
3	$128 \times 64 \times 128$	0.000844	0.000449	0.000153	$1.88\times$	$5.51\times$
4	$32 \times 128 \times 32$	0.000076	0.000048	0.000082	$1.60\times$	$0.94\times$
5	$200 \times 100 \times 256$	0.003272	0.001776	0.000863	$1.84\times$	$3.79\times$
6	$256 \times 256 \times 256$	0.012521	0.006108	0.003052	$2.05\times$	$4.10\times$
7	$256 \times 300 \times 256$	0.015248	0.007838	0.003625	$1.95\times$	$4.21\times$
8	$64 \times 128 \times 64$	0.000318	0.000181	0.000122	$1.76\times$	$2.60\times$
9	$256 \times 256 \times 257$	0.012761	0.005966	0.004022	$2.14\times$	$3.17\times$

The blocked variant achieves a speedup upto $2.2\times$ and the parallel variant reaches $5.5\times$. The smallest cases (0, 4, 8) continue to be dominated by OpenMP startup overhead in the parallel column; this is expected and is a fundamental limit of running short workloads through a thread-team primitive.

A worth noting thing to mention is that the absolute timings between runs vary on a typical WSL environment due to OS scheduling and other memory related operations. While the five-iteration averaging in the timing harness mitigates but does not eliminate the variance completely.

7 Conclusions

There were three worth noting lessons in the assignment.

First, the *order* in which loops are nested matters as much as whether blocking is applied at all. The single change from $i \rightarrow j \rightarrow k$ to $i \rightarrow k \rightarrow j$ in the inner triple, a transformation orthogonal to blocking but enabled by the same restructuring, was responsible for the largest single jump in parallel speedup and was a prerequisite for the compiler-level vectorization gains observed later.

Second, the regime in which an optimization is meant to operate determines whether it can succeed. Cache blocking is designed for matrices that exceed the cache hierarchy. Within this assignment’s test matrices, which fit entirely in L2 cache, the blocked variant could not be expected to substantially outperform the naive variant, regardless of how cleverly it was implemented. This is not a failure of the implementation; it is a property of the workload.

Third, compiler optimization flags interact strongly with algorithmic choices. The $i \rightarrow k \rightarrow j$ ordering established in § 5.3 created an inner loop of a form the auto-vectorizer can exploit, but the auto-vectorizer is only engaged at `-O3`. Until the compiler flags were investigated in § 6.2, the algorithmic improvement was leaving most of its potential value on the table.

8 Reproducibility

To reproduce results using the build path the assignment specifies:

```
cmake -DCMAKE_C_COMPILER=gcc -DCMAKE_CXX_COMPILER=g++ .  
make  
export OMP_NUM_THREADS=8  
for i in 0 1 2 3 4 5 6 7 8 9; do ./matmul $i; echo; done
```

To reproduce the results in Table 2, build directly with the optimization flags discussed in § 6.2:

```
g++ -std=c++11 -fopenmp -O3 -march=native -ffast-math -Wall main.cpp -o  
    matmul  
export OMP_NUM_THREADS=8  
for i in 0 1 2 3 4 5 6 7 8 9; do ./matmul $i; echo; done
```